



**JS**

# **Peticiones asíncronas - AJAX**

**Laboratorio de Programación**

Ing. Rasjido Jose

UNPA - UACO

# AGENDA

- ✓ **Introducción a AJAX**
- ✓ Generar una petición asíncrona
- ✓ Eventos asociados a una petición
- ✓ API Fetch
- ✓ Bibliografía

# Introducción a AJAX

## ¿Qué es AJAX?

**AJAX** (Asynchronous JavaScript and XML – JavaScript Asíncrono con XML) es un término creado en 2005 por Jesse James Garret, quien propuso un conjunto de técnicas mientras escribía un artículo para una revista.

Técnicamente es una práctica de programación utilizada para construir páginas Web dinámica e interactivas a través de peticiones asíncronas.

# Introducción a AJAX

## El objeto XMLHttpRequest (XHR)

- Permite utilizar tecnología conocida como *Remote Scripting*, sin necesidad de que el usuario tenga instalada una JVM.
- La tecnología de *Remote Scripting* tiene sus orígenes en el año 1998 y fue desarrollada por Microsoft.
- A partir de Internet Explorer 7 y para el resto de los navegadores en cualquier versión, el objeto se instancia de la clase `XMLHttpRequest`.

# AGENDA

- ✓ Introducción a AJAX
- ✓ **Generar una petición asíncrona**
- ✓ Eventos asociados a una petición
- ✓ API Fetch
- ✓ Bibliografía

# Generar una petición asíncrona

## Procedimiento para realizar una petición asíncrona

- 1) Instanciar el objeto
- 2) Configurar y abrir la petición
- 3) Definir una función JavaScript que se encargue de administrar la evolución de la petición (dada su asincronía)
- 4) Enviar la petición y los datos al servidor
- 5) En la función definida, manipular el estado de la petición y darle tratamiento a la respuesta del servidor.

# Generar una petición asíncrona

## (1) Instanciar el objeto

```
let petición = new XMLHttpRequest();
```

# Generar una petición asíncrona

## (2) Configuración

Luego de instanciar una petición, se deben configurar los siguientes métodos:

- a) `open`
- b) `setRequestHeader`
- c) `responseType`

# Generar una petición asíncrona

## (2) a) Configuración => `open()`

Permite establecer los parámetros requeridos por una petición. No abre ni inicia la petición. Sintaxis:

```
peticion.open(método, URL, asincronismo, usuario, clave);
```

# Generar una petición asíncrona

## (2) a) Configuración => Parámetros de open()

- **metodo**: es un String, en el cual se especifica el método de envío (get o post).
- **URL**: es un String con la dirección del recurso a invocar.
- **asincronismo**: es un valor lógico que indica si la petición será asíncrona (true) o sincrónica (false).
- **usuario**: opcional, en caso de necesitar autenticación con el servidor.
- **clave**: opcional.

# Generar una petición asíncrona

## (2) b) Configuración => `setRequestHeader()`

Permite definir un parámetro en el encabezado HTTP de la petición que irá al servidor. Debe definirse antes de `send()`. Sintaxis:

```
peticion.setRequestHeader(atributo, valor);
```

# Generar una petición asíncrona

## (2) c) Configuración => `responseType`

Permite definir como será interpretada la respuesta del servidor por el XHR.

El valor del atributo debe ser un *string* que indique el tipo de respuesta:

- “`arraybuffer`” => Un arraybuffer con datos binarios
- “`blob`” => Un objeto blob conteniendo datos binarios
- “`document`” => Un documento HTML o XML
- “`json`” => Un objeto JavaScript
- “`text`” | “” => Un texto en un string (por defecto)

# Generar una petición asíncrona

## (3) Definir función monitor => `onreadystatechange`

Se utiliza para definir el comportamiento del objeto ante los cambios de estados del mismo.

Se define una función que monitorice y actúe de acuerdo al estado actual de la petición. Sintaxis:

```
peticion.onreadystatechange = funcionMonitor;
```

# Generar una petición asíncrona

## (4) Envió de petición y datos al servidor => `send()`

Este método inicia la petición hacia el servidor.

Sintaxis:

```
peticion.send ( [null], [parámetro=valor] );
```

# Generar una petición asíncrona

## (5) Función monitor

Debe evaluar el estado de la petición y capturar la respuesta del servidor. Para ello se disponen de los siguientes atributos:

- a) `readyState`
- b) `status`
- c) `responseText`

# Generar una petición asíncrona

## (5) Función `monitor` => `readyState`

Devuelve un código numérico que indica el estado de la petición.

- **0 = sin inicializar.** La petición solo fue instanciada.
- **1 = cargando.** La petición se configuró, pero no se envió.
- **2 = espera.** El requerimiento se envió o se está enviando, pero todavía no hay respuesta del servidor.
- **3 = interactivo.** El servidor ya respondió, se recibió la cabecera de la petición, pero falta el contenido.
- **4 = completo.** Se recibió el mensaje HTTP completo.

# Generar una petición asíncrona

## (5) Función monitor => readyState

- 0 = XMLHttpRequest.UNSET
- 1 = XMLHttpRequest.OPENED
- 2 = XMLHttpRequest.HEADERS\_RECEIVED
- 3 = XMLHttpRequest.LOADING
- 4 = XMLHttpRequest.DONE

# Generar una petición asíncrona

## (5) Función monitor => status

Devuelve un código numérico, el cual indica si el servidor pudo o no atender la petición.

- **200**. La petición se procesó de forma correcta.
- **400**. La petición enviada al servidor es errónea.
- **403**. No se pudo acceder al recurso del servidor por falta de permisos.
- **404**. La URL solicitada no existe.
- **500**. Error interno del servidor.
- **503**. El servidor no se encuentra disponible temporalmente.

# Generar una petición asíncrona

## (5) Función monitor => `responseText`

Devuelve un String con el contenido del cuerpo del mensaje HTTP. Solo se tiene seguridad de que los datos están correctos cuando `readyState` sea 4 y `status` sea 200.

Si en ***responseType*** se especificó algo distinto a “text”, entonces se debe acceder a la propiedad ***response***.

# AGENDA

- ✓ Introducción a AJAX
- ✓ Generar una petición asíncrona
- ✓ **Eventos asociados a una petición**
- ✓ API Fetch
- ✓ Bibliografía

# Eventos asociados a una petición

XHR provee la capacidad de monitorear varios eventos que ocurren mientras una petición está siendo procesada. Los eventos provistos son:

- El evento ***load***. Este se dispara cuando la transacción se completó exitosamente.

```
addEventListener("load", (event) => {});
```

```
onload = (event) => {};
```

# Eventos asociados a una petición

- El evento **error**. Este se dispara cuando ocurren un error durante la petición.

```
addEventListener("error", (event) => {});
```

```
onerror = (event) => {};
```

- El evento **abort**. Este se dispara cuando una petición es cancelada.

```
addEventListener("abort", (event) => {});
```

```
onabort = (event) => {};
```

# Eventos asociados a una petición

- El evento **loadend**. Se dispara cuando una petición se completa, sin importar el resultado (éxito o fracaso).

```
addEventListener("loadend", (event) => {});
```

```
onloadend = (event) => {};
```

- El evento **loadstart**. Se dispara cuando una petición comienza a cargar datos.

```
addEventListener("loadstart", (event) => {});
```

```
onloadstart = (event) => {};
```

# AGENDA

- ✓ Introducción a AJAX
- ✓ Generar una petición asíncrona
- ✓ Eventos asociados a una petición
- ✓ **API Fetch**
- ✓ Bibliografía

# API Fetch

## ➤ ¿Qué es API FETCH?

Es una librería de Javascript que proporciona una interfaz para recuperar recursos.

Es una mejora de la tecnología XMLHttpRequest, que provee un conjunto de características más potente y flexible.

# API Fetch

## El método fetch()

El método fetch() toma un argumento obligatorio, la ruta de acceso al recurso que desea recuperar.

Devuelve una Promesa que resuelve una respuesta (response) a esa petición, sea o no correcta.

# API Fetch

## Ejemplo de una petición fetch()

```
fetch('http://example.com/movies.json')  
  
  .then(response => response.json())  
  
  .then(data => console.log(data));
```

Aquí estamos recuperando un archivo JSON a través de internet, e imprimiendo los datos en la consola.

# API Fetch

## Comprobando que la petición es satisfactoria

Una forma precisa de comprobar que la petición `fetch()` es satisfactoria, es verificar si la promesa ha sido resuelta y el atributo “`ok`” de la respuesta tiene el valor `true`. Es decir que el estado de la petición HTTP es OK (código 200-299).

# API Fetch

## Ejemplo

```
.then(function (response) {  
    if (response.ok) {  
        //Procesar la respuesta  
    }  
} else {  
    console.log("Respuesta de red OK pero respuesta HTTP no OK");  
}  
}))  
  
.catch(function (error) {  
    console.log("Hubo un problema con la petición Fetch:" + error.message);  
});
```

# API Fetch

Ejemplo: Combo/listas dependientes.

Descripción: Cargar las provincias argentinas, luego cargar las localidades de la provincia seleccionada.

API a consumir: <https://www.argentina.gob.ar/datos-abiertos/georef/openapi>

index.html

```
<select id="selectProvincias"></select>
```

```
<select id="selectLocalidades"></select>
```

# API Fetch

script.js

```
const fetchProvincias = ()=>{  
    fetch('https://apis.datos.gob.ar/georef/api/provincias?orden=nombre')  
    .then(response => {  
        if(response.ok) return response.json();  
        throw new Error(response, response.status);  
    })  
    .then(data => { //cargar las provincias  
        cargarProvincias(data.provincias);  
    })  
    .catch(error => { console.error("Ha ocurrido un error", error); });  
}
```

# API Fetch

script.js

```
const fetchLocalidades = (idProvincia)=>{

    fetch('https://apis.datos.gob.ar/georef/api/localidades-censales?provincia=' + idProvincia +
    '&max=2000' + '&orden=nombre')

        .then(response => {

            if(response.ok) return response.json();

            throw new Error(response, response.status);

        })

        .then(data => { //cargar las localidades

            cargarLocalidades(data.localidades_censales);

        })

        .catch(error => { console.error("Ha ocurrido un error", error); });

}
```

# API Fetch

script.js

```
const cargarProvincias = provincias => {  
    let select = document.getElementById("selectProvincias");  
    provincias.forEach(prov => {  
        select.add(new Option(prov.nombre, prov.id));  
    });  
  
    select.onchange = (event) => {  
        fetchLocalidades(parseInt(event.target.value));  
    }  
}
```

# API Fetch

script.js

```
const cargarLocalidades = localidades => {  
    let select = document.getElementById("selectLocalidades");  
    select.innerHTML = "";  
    localidades.forEach(loc => {  
        select.add(new Option(loc.nombre, loc.id));  
    });  
}
```

# API Fetch

script.js

```
document.addEventListener("DOMContentLoaded", ()=>{  
    fetchProvincias();  
})
```

# Bibliografía

- **Living Estándar WHATWG**
  - <https://xhr.spec.whatwg.org/>
  - <https://xhr.spec.whatwg.org/#interface-xmlhttprequest>
- **Documentación de Mozilla**
  - [https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest\\_API/Using\\_XMLHttpRequest](https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest_API/Using_XMLHttpRequest)